

Zrównoleglenie algorytmu HGS dla problemu CVRP

Wpływ akceleracji GPU, modelu master–slave i modelu wysp na jakość rozwiązań

Hubert Błonowski Jan Zaborski

14 czerwca 2026

Plan prezentacji

Problem CVRP

Algorytm HGS

Wprowadzone modyfikacje

Wyniki eksperymentów

Wnioski

Czym jest CVRP?

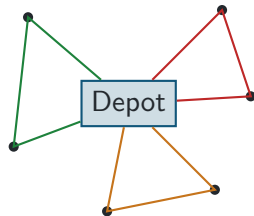
Capacitated Vehicle Routing Problem — kanoniczny problem optymalizacji tras dostaw.

Dane wejściowe:

- ▶ jeden magazyn (*depot*) oraz n klientów,
- ▶ zapotrzebowanie q_i każdego klienta,
- ▶ flota pojazdów o wspólnej pojemności Q ,
- ▶ macierz odległości (kosztów) między punktami.

Cel: zbiór tras (każda od depotu do depotu) o **minimalnej łącznej długości**, taki że:

- ▶ każdy klient odwiedzony dokładnie raz,
- ▶ suma zapotrzebowań na trasie $\leq Q$.



Trzy trasy obsługujące sześciu klientów.

Dlaczego to trudny problem?

- ▶ CVRP jest **NP-trudny** — uogólnia problem komiwojażera (TSP).
- ▶ Liczba możliwych podziałów i kolejności rośnie wykładniczo z liczbą klientów.
- ▶ Dla instancji z tysiącami klientów metody dokładne są bezużyteczne — stosuje się **metaheurystyki**.
- ▶ Znaczenie praktyczne: logistyka, kurierzy, dystrybucja, odbiór odpadów — poprawa o ułamek procenta na dużej flocie to realne oszczędności paliwa i czasu.

Punkt odniesienia

HGS (Hybrid Genetic Search, Vidal i in.) to jeden z najlepszych znanych algorytmów dla CVRP — bardzo dobre rozwiązania w krótkim czasie. Naszą pracę budujemy jako jego rozszerzenie (fork).

HGS w skrócie: ewolucja + intensywna lokalna poprawa

HGS łączy **algorytm genetyczny** (eksploracja przestrzeni) z **przeszukiwaniem lokalnym** (eksploatacja okolicy rozwiązania). Pięć współpracujących komponentów:

1. **Reprezentacja + Split** — rozwiązanie jako permutacja klientów, dzielona na trasy.
2. **Krzyżowanie (OX)** — z dwóch rodziców powstaje potomek.
3. **Przeszukiwanie lokalne** — RI (relokacje, zamiany, 2-opt) oraz SWAP*.
4. **Populacja z różnorodnością** — podpopulacje dopuszczalna/niedopuszczalna, selekcja wg *biased fitness*.
5. **Adaptacyjne kary** — sterują dopuszczalnością przez całą optymalizację.

Kluczowa cecha: pętla jest **sekwencyjna i adaptacyjna** — każda iteracja zależy od stanu poprzednich. To później okaże się istotne dla zrównoleglenia.

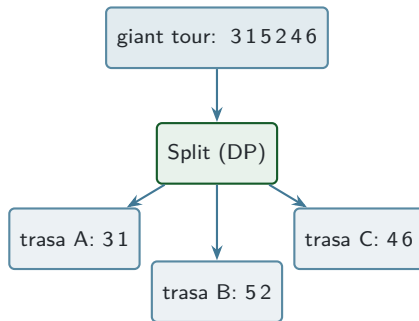
Reprezentacja rozwiązania i algorytm Split

Gigantyczna trasa (*giant tour*). Rozwiązanie to **permutacja klientów** $1 \dots n$ — sama kolejność odwiedzin, *bez* przypisania do pojazdów. Krzyżowanie działa właśnie na tej permutacji.

Split. Optymalnie tnie permutację na trasy metodą **programowania dynamicznego** (Bellman po grafie acyklicznym):

- ▶ wariant $O(n)$ bez ograniczeń czasowych (Vidal 2016),
- ▶ wariant z limitem floty (`splitLF`) i z ograniczeniem czasu trasy.

Wynik: konkretne trasy + koszt z naruszeniami.



Split rozdziela koszt krzyżowania od struktury tras.

Po Splicie potomek jest intensywnie ulepszany. Rozwiązanie trzymane jest jako **lista dwukierunkowa** węzłów (Node) i tras (Route) — szybkie wstawianie i usuwanie klientów.

Ruchy RI (klasyczny zestaw Prinsa, move1–move9):

- ▶ relokacja 1 lub 2 klientów,
- ▶ zamiana klientów (swap),
- ▶ 2-opt (wewnątrz trasy) i 2-opt* (między trasami).

Dwa mechanizmy przyspieszające:

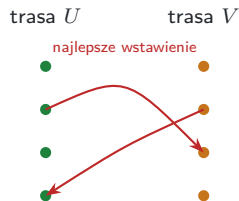
- ▶ **Sąsiedztwo granularne** — ruch rozważany tylko między klientem a jego nbGranular najbliższymi (correlatedVertices).
- ▶ **„Don't-look”** — znaczniki whenLastTested + licznik nbMoves pomijają ponowną ocenę niezmienionych węzłów.

Local search jest **najdroższym krokiem** całej tury — i głównym celem naszego zrównoleglenia.

Sąsiedztwo SWAP* — to akcelerujemy na GPU

Dla **pary tras** (U, V) SWAP* rozważa wymianę klienta z U na klienta z V — ale **każdy z nich wstawiany jest w najlepsze miejsce** drugiej trasy (nie w pozycję wymienianego).

- ▶ ThreeBestInsert — trzy najtańsze pozycje wstawienia każdego klienta (z pominięciem sąsiedztwa usuwanego).
- ▶ **Przycinanie**: rozważane tylko pary tras o **nakrywających się sektorach kątowych** (CircleSector).
- ▶ Koszt ruchu = usunięcia + najtańsze wstawienia + zmiana kar.



Liczba ocen rośnie z **kwadratem** długości trasy — stąd masa pracy dla GPU.

Dwie podpopulacje, każda sortowana wg kosztu z karami:

- ▶ **dopuszczalna** (spełnia pojemność/czas),
- ▶ **niedopuszczalna** (z naruszeniami).

Biased fitness łączy dwa rankingi:

$$f_{\text{bias}} = \underbrace{r_{\text{koszt}}}_{\text{jakość}} + \left(1 - \frac{n_{\text{elite}}}{|P|}\right) \underbrace{r_{\text{div}}}_{\text{różnorodność}}$$

Różnorodność = *broken-pairs distance* do nbClose najbliższych rozwiązań.

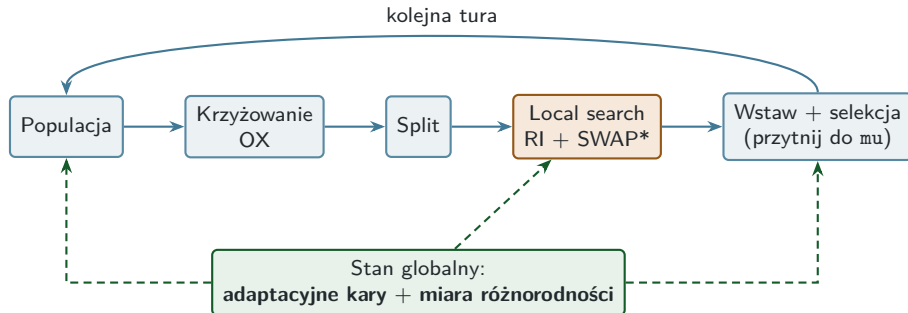
Selekcja ocalałych. Gdy podpopulacja przekroczy $\mu + \lambda$, przycinana jest **do** μ : usuwany najgorszy wg biased fitness, z **preferencją usuwania klonów**.

Po co różnorodność?

Sama jakość prowadzi do przedwczesnej zbieżności (klony). Premiowanie odmiennych rozwiązań utrzymuje eksplorację.

- ▶ **Kary** `penaltyCapacity` i `penaltyDuration` są adaptowane co `nbIterPenaltyManagement` iteracji w stronę docelowego odsetka rozwiązań dopuszczalnych (`targetFeasible` $\approx 0,2$): za mało dopuszczalnych $\Rightarrow$ kary rosną, za dużo $\Rightarrow$ maleją. Pozwala to kontrolowanie naruszeń *przejsciowo*.
- ▶ **Krzyżowanie OX** (ordered crossover) na gigantycznych trasach: kopiuje wycinek pierwszego rodzica, resztę uzupełnia w kolejności drugiego.
- ▶ **Zakończenie / restart**: brak poprawy przez `nbIter` iteracji kończy bieg; przy limicie czasu populacja jest restartowana i optymalizacja trwa do wyczerpania czasu.

Pełna pętla HGS



Przerywane strzałki: globalny, **współdzielony stan** adaptowany w trakcie biegu — źródło zależności danych utrudniających zrównoleglenie.

Trzy osie zrównoleglenia

Do HGS dodaliśmy trzy niezależne (i łączalne) mechanizmy:

1. GPU SWAP* (CUDA)

Ewaluacja par tras dla SWAP* przeniesiona na GPU; jeden blok na parę, strumienie CUDA.

2. Master-slave (OpenMP)

Równoległe wytwarzanie potomstwa: 16 potomków na rundę na 8 wątkach.

3. Model wysp (MPI)

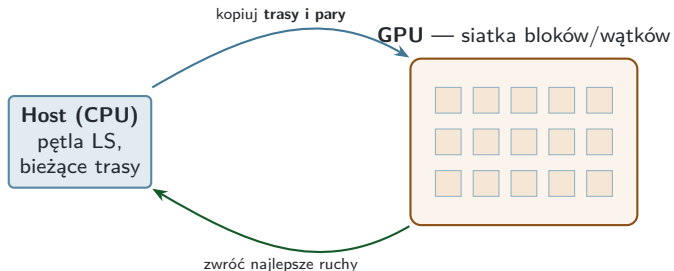
8 niezależnych populacji, topologia gwiazdy, polityki migracji i selekcji migrantów.

Dodatkowo: warstwa **telemetrii** i **monitor sprzętu** (CPU/GPU, NVML) do rzetelnej oceny.

Sensownie przetestowaliśmy **6 wariantów**: baseline, gpu, offspring16t8, gpu_offspring16t8, island8_star, island8_star_gpu_off16t8.

Oś 1: SWAP* na GPU — pomysł

Co przyspieszamy? W każdym przebiegu local search SWAP* ocenia **ogromną liczbę par klientów** z różnych tras. To wiele niezależnych, identycznych obliczeń — *idealny wzorzec dla GPU*, które liczy tysiące takich par jednocześnie.



Jeden **blok GPU** przetwarza jedną parę tras; CPU dostaje gotowy zestaw najlepszych zamian do zastosowania.

Co zbudowaliśmy (CUDA):

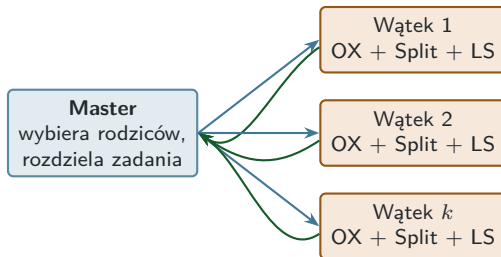
- ▶ **Kernel** — jeden blok CUDA na parę tras (256 wątków); wątki oceniają kandydatów SWAP*/relokacji, a **redukcja warp + shared memory** wyłania najlepszy ruch pary.
- ▶ **Spłaszczony układ danych** (GpuDataLayout) — trasy, klienci, koszty usunięć i pary w jednym buforze (koalescencja dostępu).
- ▶ **Współdzielony kontekst** (GpuProblemData) — macierz odległości wgrana **raz na proces** (read-only), referencja we wszystkich instancjach.
- ▶ **Strumień CUDA na instancję** — asynchroniczne transfery i synchronizacja *własnego* strumienia $\Rightarrow$ równoległe instancje LS **nakładają się** na GPU. Pomiar zużycia przez NVML.

Oś 1: SWAP* na GPU — kompromis

- ▶ Karta jest wydajna tylko przy **dużej ilości równoległej pracy na jedno wywołanie**. Liczba par rośnie z kwadratem długości trasy, więc **krótkie trasy** (dużo pojazdów) dają mało pracy i dominuje **narzut uruchomienia kernela i transferu**.
- ▶ **Realny zysk** pojawia się dopiero przy długich trasach lub wielu równoległych ocenach naraz — co pokażą wyniki.

Oś 2: model master-slave — pomysł

Co przyspieszamy? Najdroższy krok tury to local search potomka. Skoro mamy wiele rdzeni CPU — **produkujemy wielu potomków naraz**, każdy ulepszany na osobnym wątku.



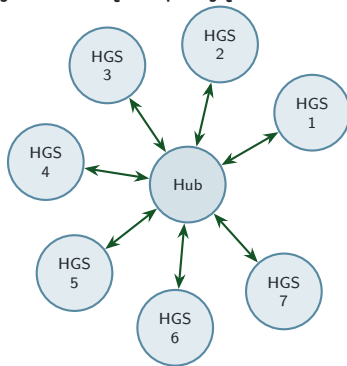
Nasz wariant: **16 potomków na rundę, 8 wątków**. Master zbiera potomków i seryjnie wstawia do populacji.

Co zbudowaliśmy (OpenMP):

- ▶ Wspólny interfejs `OffspringMaker` z dwiema implementacjami: `Standard` (sekwencyjna) i `Parallel`.
- ▶ `ParallelOffspringMaker` — pula wątków; każdy wątek wykonuje pełny cykl: **selekcja turniejowa** → **OX** → **Split** → **local search**, na **własnym** `LocalSearch` i `Split` (brak współdzielenia stanu, brak wyścigów).

Oś 3: model wysp — pomysł

Inne podejście do równoległości: zamiast jednej populacji uruchamiamy **kilka niezależnych HGS** („wysp”) obok siebie. Co jakiś czas wyspy **wymieniają najlepsze rozwiązania** (migracja), wzajemnie się inspirując.



Topologia **gwiazdy**: 8 wysp wymienia migrantów przez węzeł centralny. Każda wyspa = **pełny HGS** na własnej podpopulacji, z przesuniętym ziarnem losowym.

Oś 3: model wysp — implementacja

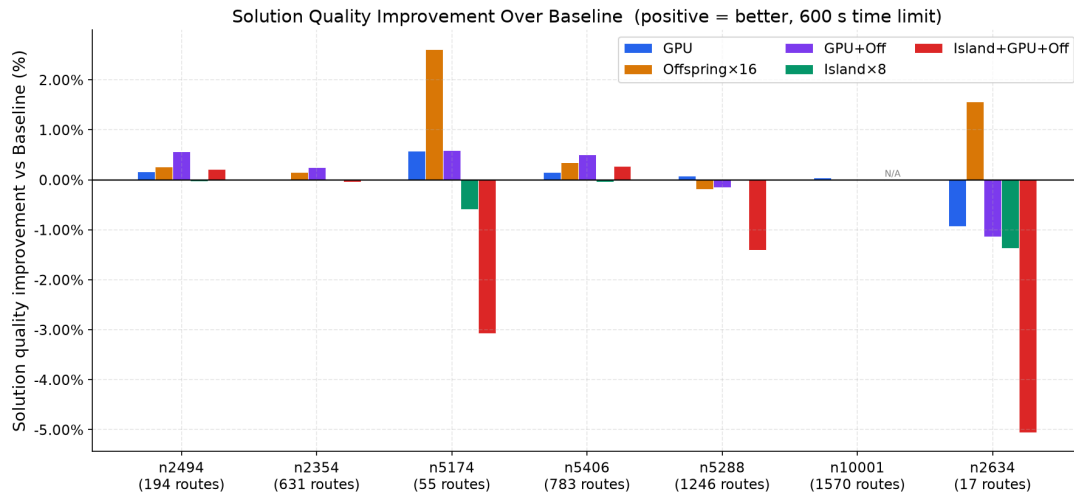
Co zbudowaliśmy (MPI, modularnie): całość rozbita na wymienne komponenty, łączone przez konfigurację:

- ▶ **Topologie** połączeń: pierścień, dwukierunkowy pierścień, hiperkostka, **gwiazda**, graf pełny, losowy regularny.
- ▶ **Polityki migracji:** stały interwał, wyzwalana poprawą, adaptacyjna, sterowana różnorodnością.
- ▶ **Selektor migrantów i obsługa imigrantów** (wstaw / z LS / z naprawą).
- ▶ **Komunikator MPI.**

Migrowana jest **gigantyczna trasa**; odbiorca odtwarza struktury tras własnym Splitem. W eksperymentach: topologia gwiazdy, 8 wysp, komunikator synchroniczny.

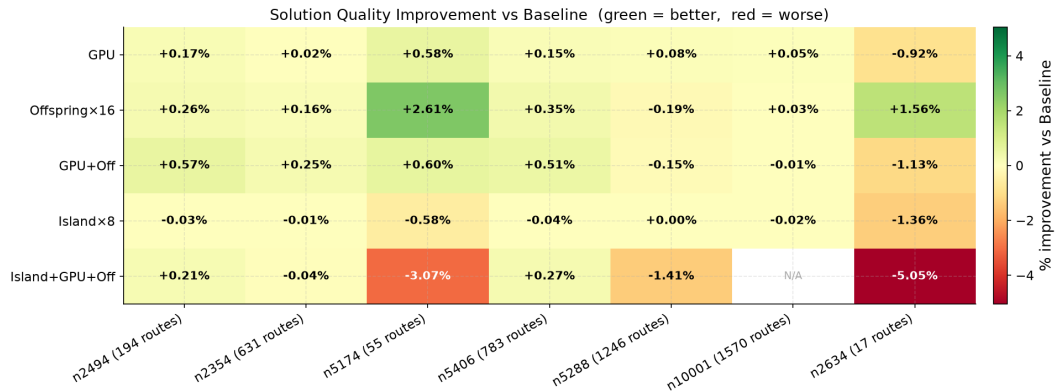
- ▶ **Instancje klasy XL** (od 2354 do 10001 klientów), **6 wariantów**, **limit czasu 600 s**, ziarno 1.
- ▶ Mierzone: jakość (koszt), liczba tur, ewaluowane pary SWAP*, ruchy lokalne, wykorzystanie CPU/GPU, migracje, rundy potomstwa.
- ▶ Cel: czy któraś oś zrównoleglenia **poprawia jakość** w stałym budżecie czasu.

Poprawa jakości względem baseline



Wartość dodatnia = lepiej niż baseline. Skala efektu: **ułamki procenta**.

Mapa cieplna jakości



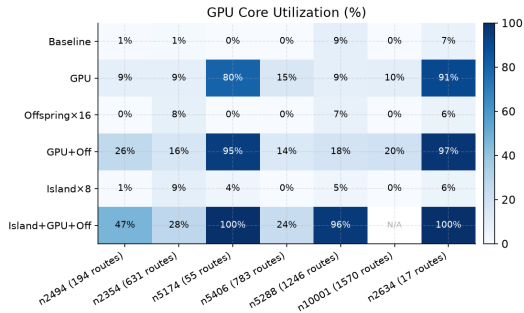
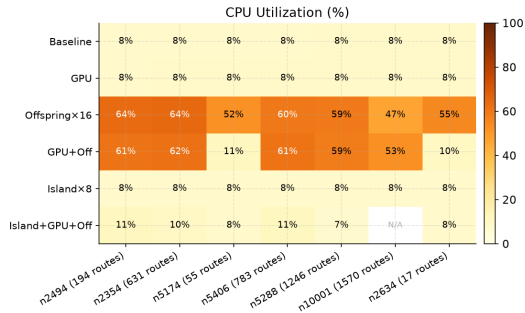
Najbardziej złożony wariant (wyspy + GPU + offspring) jest często **czerwony** — gorszy od baseline. Składanie mechanizmów nie sumuje korzyści.

Najlepszy wariant na każdej instancji

Instancja	Baseline	Najlepszy wariant	Δ	Wariant
XL-n2354-k631	947 790	945 389	+0,25%	GPU+Off
XL-n2494-k194	366 131	364 057	+0,57%	GPU+Off
XL-n2634-k17	33 220	32 701	+1,56%	Offspring
XL-n5174-k55	65 467	63 756	+2,61%	Offspring
XL-n5288-k1246	1 984 069	1 982 432	+0,08%	GPU
XL-n5406-k783	1 058 333	1 052 987	+0,51%	GPU+Off
XL-n10001-k1570	2 363 410	2 362 345	+0,05%	GPU

Nawet najlepszy wariant poprawia baseline jedynie o ułamki procenta (poza dwiema instancjami z bardzo długimi trasami).

Hardware Utilization per Preset and Instance



Utylizacja GPU rośnie przy małej liczbie pojazdów

Karta jest wydajna tylko, gdy ma **dużo równoległej pracy na jedno wywołanie**.

- ▶ SWAP* porównuje **pary klientów** w obrębie pary tras — liczba par rośnie z **kwadratem długości trasy**.
- ▶ **Mało pojazdów** $\Rightarrow$ trasy **długie** $\Rightarrow$ duże, gęste bloki pracy $\Rightarrow$ **rdzenie GPU zajęte**.
- ▶ **Dużo pojazdów** $\Rightarrow$ trasy **krótkie** (3–4 klientów) $\Rightarrow$ pracy jak na lekarstwo $\Rightarrow$ dominuje narzut launchu i transferu $\Rightarrow$ **słaba utylizacja**.

Instancja	klienci /trasę	rdzenie GPU
XL-n2634-k17	155	91%
XL-n5174-k55	94	80%
XL-n2494-k194	13	9%
XL-n5406-k783	7	15%
XL-n5288-k1246	4	9%
XL-n2354-k631	4	9%

„k” w nazwie = liczba pojazdów; średnia długość trasy = n/k .

Główny wniosek

Zrównoleglenie **nie poprawiło jakości** w istotny sposób, a najbardziej złożony wariant często ją pogarszał. Wąskim gardłem nie jest przepustowość obliczeń, lecz **sam algorytm i struktura zależności danych**.

- ▶ HGS jest zbudowany wokół **sekwencyjnej, adaptacyjnej pętli** (kary, różnorodność, selekcja), która nie rozbija się na niezależne kawałki bez zmiany dynamiki przeszukiwania.
- ▶ GPU pomaga **tylko** tam, gdzie trasy są długie (dużo równoległej pracy); przy krótkich trasach narzut przewyższa zysk.
- ▶ Lepszym kierunkiem byłyby algorytmy iteracyjnej poprawy **zaprojektowane z myślą o równoległości**, zamiast doklejania równoległości do z gruntu sekwencyjnego schematu.

Dziękujemy za uwagę.

Hubert Blonowski ■ Jan Zaborski